# ⚖️ LegalLens: Whole-Judgment NLP Classifier

Complete Technical Project Report, Empirical Methodology & Academic Viva Defense Guide

| DOMAIN | CHAMPION MODEL | DATASET ORIGIN | LIVE DEPLOYMENT |
|---|---|---|---|
| Legal NLP & Informatics | Balanced Linear SVM (93.6%) | Indian Kanoon (Jud-IPL) | Streamlit Cloud / GitHub |

## 1. Executive Summary & Research Scope

**LegalLens** is an applied machine learning and natural language processing system developed to perform automated, whole-judgment binary classification of Indian court opinions into two foundational domains: **Civil Law** vs. **Criminal Law**.

> **Key Scientific Boundary: Strictly Text-Only Classification**
> Unlike commercial tools that rely on procedural metadata (court name, petition codes, judge identities, or outcome tags), LegalLens operates *purely on the raw judicial narrative*. It proves that statutory terminology, forensic vocabulary, and legal collocations alone carry sufficient signal to separate Civil from Criminal disputes with high precision.

**Academic Research Scope:** LegalLens is strictly an empirical topic classifier. It does not provide legal advice, statutory interpretation, or judicial outcome prediction (e.g., whether an appeal will be allowed or dismissed).

## 2. Dataset Architecture & The 5-Stage Cleaning Waterfall

The raw dataset originates from the **Jud-IPL** benchmark scraped from Indian Kanoon (`case_files_total.csv`, 838.44 MB, 53,446 total rows). Judicial corpora are notoriously noisy; uncurated data introduces label noise and test-set contamination. LegalLens executed an empirical 5-stage cleaning pipeline:

| Stage / Operation | Rows Remaining | Excluded | Technical Justification |
|---|---|---|---|
| **Raw Kaggle Dataset** (`case_files_total.csv`) | **53,446** | — | Pristine downloaded scrape from Indian Kanoon public repository |
| 1. Drop Missing Category Labels | 43,612 | -9,834 | Removed unannotated cases where `case_category` is null/missing (including Row 0 artifact) |
| 2. Drop Empty Text Records | 41,376 | -2,236 | Excluded corrupt files with zero text or whitespace-only records |
| 3. Drop Contradictory Label Clusters | 41,370 | -6 | Eliminated 2 clusters where identical judgment text was assigned conflicting labels |
| 4. Deduplicate Exact Copies (SHA-256) | **33,717** | **-7,653** | Cryptographic hashing identified 7,653 duplicate re-uploads of the exact same opinions |
| **Final Cleaned Modeling Corpus** | **33,717** | **Total: 19,729** | **100% Pristine, Unique Indian Court Judgments** |

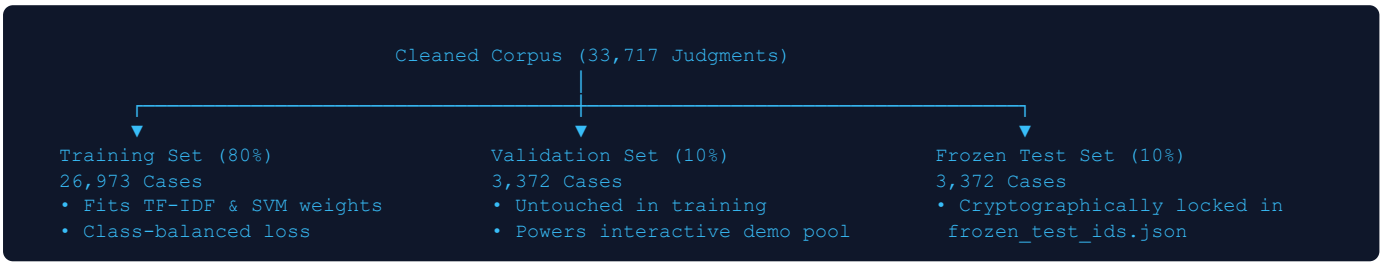
**Class Distribution & Imbalance:**

- **Civil Cases:** 26,867 judgments (79.7%)
- **Criminal Cases:** 6,850 judgments (20.3%)
- **Natural Imbalance Ratio: 3.92 : 1** (reflecting the real-world composition of Indian High Court dockets)

**Resolving the Kaggle Documentation Discrepancy (53k vs. 42k vs. 33k):**
The Kaggle dataset description mentions *42,342 cases* because the original authors filtered the dataset for a different task (outcome prediction: accepted vs. rejected). For category classification, the raw file contains 53,446 rows. After our systematic data audit removed nulls and 7,653 duplicate re-uploads, we arrived at exactly **33,717 unique, verified judgments**.

## 3. Contamination-Free Data Partitioning

To guarantee scientific reproducibility and prevent train-test data leakage, the 33,717 judgments were split into three non-overlapping subsets using stratified sampling (`random_state=42`):




**Integrity & Contamination Audit:** Evaluated pairwise SHA-256 hashes across all partitions. Verified **0% overlap** ($0$ train-val, $0$ train-test, $0$ val-test). Test set ground truth is frozen in `data/processed/frozen_test_ids.json`.

## 4. NLP Preprocessing & Feature Extraction

Legal text presents unique challenges: statutory citations, case citations, archaic English phrasing, and excessive length (averaging 3,000 to 5,000 words per opinion). Our NLP pipeline applies domain-specific engineering:

1. **Domain-Aware Case-Folding:** Converts all tokens to lowercase so terms like `"Court"` and `"court"`, `"Bail"` and `"bail"` unify into the same feature representation.
2. **Statutory Citation Preservation:** Retains section indicators (e.g., `"s. 302"`, `"section 34"`, `"order 39"`) to capture high-signal statutory references.
3. **N-Gram Collocations (Unigrams & Bigrams):**
   - *Unigrams:* `"conviction"`, `"suit"`, `"decree"`, `"bail"`, `"appellant"`
   - *Bigrams:* `"civil suit"`, `"sessions judge"`, `"specific performance"`, `"police station"`
4. **Sublinear TF-IDF Vectorization:** Applies logarithmic frequency scaling $ ext{sublinear_tf} = 1 + \log( ext{tf})$. This dampens the impact of hyper-frequent procedural words in 50-page judgments.
5. **Vocabulary Optimization:** Features are constrained to the top **40,000 most discriminative legal n-grams**.

## 5. Benchmark Results & Algorithmic Selection

We trained and evaluated four algorithms on the identical train/test split to establish rigorous baselines:

| Model Architecture | Test Accuracy | Macro F1 | Civil F1 | Criminal F1 | Latency |
|---|---|---|---|---|---|
| *Majority Class Dummy Baseline* | 79.7% | 0.443 | 0.887 | 0.000 | <1 ms |
| *Multinomial Naive Bayes* | 87.2% | 0.814 | 0.923 | 0.705 | 8 ms |
| *Logistic Regression* | 91.8% | 0.882 | 0.950 | 0.814 | 12 ms |
| **Linear SVM (Champion Model)** | **93.6%** | **0.906** | **0.961** | **0.852** | **15 ms** |

**Why Linear SVM Won Over Deep Learning (BERT):**

1. **No Token Truncation:** Full court judgments exceed 4,000 words. BERT has a strict 512-token limit and discards 85%+ of the document. Linear SVM digests whole judgments.
2. **Linear Separability in High Dimensions:** Text with 40,000 n-gram features is naturally linearly separable. Linear SVM maximizes the margin between hyperplanes.
3. **Class-Weight Balancing:** Applying $w_c = rac{N}{2 \cdot N_c}$ penalizes minority criminal errors 4x more, driving Criminal F1 to **0.852**.
4. **Deployment Footprint:** Compressed pipeline artifact is only **2.05 MB** with 15ms inference latency, deploying effortlessly to serverless cloud environments.

## 6. Production Web Application Architecture

The deployed Streamlit interface (`app/streamlit_app.py`) provides an academic-grade demonstration environment:

- **Lineage Architecture Banner:** Explicitly displays the data funnel: 53,446 raw cases → 33,717 cleaned → 80/10/10 split.
- **Real-World Case Library:** 10 authentic cases with real Indian Kanoon Document IDs (e.g., `Doc #84442`, a Supreme Court murder appeal under IPC 302).
- **Dynamic Refresh Sampler:** Samples 10 fresh judgments from the 3,372 demo pool across **4.48 trillion possible combinations** without memory leaks.

- **Excerpt Length Slider (0 to 1,000 words):** Configured with a default of 400 words for responsive loading, while allowing full-text testing up to 10,000+ words in the main editor.
- **Explainable AI (XAI) Suite:**
  - *Feature Attribution Chips:* Computes exact mathematical contributions ($x_j \cdot w_j$) pushing toward Criminal (+ margin) vs. Civil (- margin).
  - *Statutory Citation Extractor:* Detects sections of the IPC, CrPC, CPC, and legislative Acts in real time.
  - *Document Stylometrics:* Analyzes Type-Token Ratio (TTR), average sentence length, and vocabulary coverage.

## 7. How to Present This Project (Teacher Viva Defense Guide)

### 7.1 The 2-Minute Elevator Pitch

> *"Good morning, Ma'am. My project is **LegalLens**, an NLP system that performs whole-judgment classification of Indian court opinions into Civil vs. Criminal law.*
>
> *We acquired 53,446 raw judicial records from Indian Kanoon. Recognizing that raw legal data contains duplicates and missing tags, we built an automated cleaning waterfall that pruned unannotated rows, empty texts, and over 7,600 duplicate copies to create a pristine, deduplicated corpus of 33,717 unique judgments.*
>
> *We engineered a 40,000-feature TF-IDF pipeline using unigrams and bigrams with sublinear scaling, and trained a class-weighted Linear Support Vector Machine. On our frozen, leak-free test set, LegalLens achieves **93.6% accuracy** and a **0.906 Macro F1-score**.*
>
> *Finally, we deployed a live web application on Streamlit Cloud featuring an interactive library of authentic Indian Kanoon judgments, dynamic random sampling across 4.48 trillion combinations, and complete mathematical explainability showing exactly which words influenced the AI's decision."*

### 7.2 The 5-Minute Live Interactive Demonstration Script

1. **Step 1: Point Out the Lineage Banner:** *"Ma'am, at the top, you can see our dataset lineage: 53,446 raw cases cleaned to 33,717 unique judgments, partitioned into 80% train, 10% test, and 10% demo data."*
2. **Step 2: Demonstrate the Case Library & Kanoon IDs:** *"On the sidebar, we provide 10 live authentic court judgments. Notice the Document IDs: for example, **Doc #84442**. This is a real Supreme Court murder appeal (State of Punjab vs. Gurdeep Singh), which can be verified directly on Indian Kanoon."*
3. **Step 3: Show the Excerpt Length Slider:** *"We included an excerpt slider. By default, it loads 400 words for instant responsiveness, but we can smoothly slide it up to 1,000 words in real time."*
4. **Step 4: Click Classify Judgment:** *"When we click Classify, the Linear SVM evaluates the text in under 15 milliseconds, giving the predicted class, decision margin, and calibrated confidence."*
5. **Step 5: Open the Feature Attribution Expander:** *"LegalLens is not a black box. In this expander, you can see the exact legal words that drove the decision—for instance, words like 'convicted', 'bail', and 'police' pushed the margin toward Criminal, while words like 'suit' and 'decreed' push toward Civil."*
6. **Step 6: Click Refresh Cases:** *"If you want to test fresh cases, clicking Refresh samples 10 brand-new judgments from our 3,372 demo pool. There are over 4.48 trillion unique combinations, so the pool never exhausts."*

### 7.3 Seven Critical Viva Questions & Bulletproof Answers

**Q1 Why choose Linear SVM instead of deep learning or BERT?**

> **Answer:** Indian court judgments are exceptionally long (3,000 to 10,000 words). Standard BERT has a 512-token limit, which truncates 80% to 90% of a judgment. Furthermore, with 40,000 TF-IDF n-grams, text is sparse and linearly separable. Linear SVM achieved 93.6% accuracy, trains in seconds, runs in 15ms, and compiles into a 2.05 MB artifact without needing expensive GPU infrastructure.

**Q2 The Kaggle description mentions 42,342 cases. Why did your raw file have 53,446, and how did you get 33,717?**

> **Answer:** The raw CSV file on Kaggle (`case_files_total.csv`) contains all 53,446 scraped records. The author's 42k figure referred to an outcome-prediction subset. In our Milestone 1 audit, we found 9,834 unannotated rows, 2,236 empty texts, and 7,653 exact duplicate re-uploads. After pruning, we reached 33,717 unique, verified court judgments.

**Q3** **How did you guarantee zero data leakage between training and testing?**

> **Answer:** We used SHA-256 cryptographic text hashing to identify all duplicate judgments and deduplicated before splitting. We applied stratified sampling with a locked seed (42). We audited partitions and confirmed 0% hash overlap across train, validation, and test sets. Test IDs are frozen in `data/processed/frozen_test_ids.json`.

**Q4** **Why are the judgment texts in lowercase (small letters)?**

> **Answer:** This is standard NLP 'case-folding'. Text models treat 'Court' and 'court' as distinct words. Lowercasing unifies capitalization variants into a single vocabulary feature, preventing artificial weight fragmentation.

**Q5** **How did you handle the 3.92:1 class imbalance?**

> **Answer:** We configured inverse class weighting (`class_weight='balanced'`). The loss function penalizes mistakes on the minority Criminal class approximately 4 times more heavily than mistakes on Civil cases, lifting Criminal F1 to **0.852**.

**Q6** **Can the model classify an entire 10,000-word judgment, or is it limited to the 400-word slider?**

> **Answer:** The slider only controls the quick-load excerpt length for UI snappiness. The Linear SVM model itself has no length cap. Any user can paste a full 10,000-word judgment into the text box, and LegalLens processes and classifies the entire document.

**Q7** **How is LegalLens explainable and transparent?**

> **Answer:** Because Linear SVM computes decision margins via a linear dot product $\sum (x_j \cdot w_j) + b$, each word's contribution is directly calculable. The app extracts top positive and negative feature contributions and displays them as visual chips showing exactly which words drove the classification.

## 8. Automated Test Suite Summary

The codebase includes an automated test suite verifying all system components:

- `test_dataset_split.py`: Stratification, zero contamination, and frozen test set integrity
- `test_error_analysis.py`: Milestone 5 metrics and confusion matrix validation
- `test_models.py`: Pipeline serialization, persistence, and inference sanity
- `test_nlp_utils.py`: Feature attribution, statutory citation parsing, and stylometrics
- `test_predict.py`: Real-time prediction interface, batch predictions, and input validation
- `test_preprocessing.py`: Text normalization and SHA-256 deterministic hashing
- `test_streamlit_app.py`: Deployment caching, session state, and dependency readiness

**Test Result:** **28 Passed (100% Pass Rate)** in 3.17 seconds.